MODULE 13 · DOMAIN 2 - RELIABILITY AND BUSINESS CONTINUITY

Load Balancing and Its Error Codes

ALB, NLB, target groups – and reading 502, 503 and 504 as a map of where the request died.

DOMAIN 2 of 5 EXAM WEIGHT 22% MODULE 13 of 35

admin@sthithapragna.com

The load balancer sits in the middle of three hops. **The error code tells you which hop broke**, before you look at anything else.

Four load balancers

TYPE	WHAT IT IS FOR
Application Load Balancer	Layer 7. HTTP and HTTPS. Content-based routing, authentication, WAF
Network Load Balancer	Layer 4. TCP, UDP, TLS. Static IPs, extreme throughput, source IP preserved
Gateway Load Balancer	Layer 3. Transparently inserts third-party appliances using GENEVE on port 6081
Classic Load Balancer	Previous generation. Migration target, not a choice for new work

ON THE EXAM

Only ALB parses HTTP, so only ALB can route on a path or a header. Only NLB gives you a static IP. Only GWLB inserts a firewall transparently.

ALB or NLB?

ALB WHEN

HTTP intelligence

- Route by path, host, header, query or method
- Authenticate with Cognito or OIDC
- WAF attaches directly
- Lambda and IP targets, weighted target groups

NLB WHEN

Speed and addresses

- Static IP per AZ, or bring your own Elastic IP
- Non-HTTP protocols: TCP, UDP, TLS
- Client source IP preserved to the target
- Millions of requests per second, ultra-low latency

THE TIEBREAKER

Need static IPs and path routing? Put an ALB behind an NLB, using an ALB-type target group. That is a supported pattern, not a hack.

Listeners, rules and targets

- A **listener** watches a port and protocol. **Rules** are evaluated in **priority order**, and the **default rule** catches everything else.
- Conditions: **host header, path, HTTP header, query string, source IP, HTTP method**. Actions: forward, redirect, fixed response, authenticate.
- Target types: **instance**, **IP**, **Lambda**, and **ALB** (as a target of an NLB).
- **IP targets are required** for targets outside the VPC, reachable over peering, VPN or Direct Connect.

> ON THE EXAM
>
> Rules are first match wins, in priority order. A broad rule at priority 1 silently shadows every specific rule beneath it.

Target group health checks

- Protocol, port, path, **interval**, **timeout**, **healthy threshold**, **unhealthy threshold**, and a **success matcher** such as 200 or 200-299.
- The health check runs from the load balancer, so the **target's security group must allow the load balancer's security group** on the health check port.
- Point the check at a **dedicated endpoint** that verifies dependencies, not at the home page. A 200 from a static page proves nothing.
- **Target group health** lets you set a minimum healthy count or percentage, and what the load balancer does when that is not met.

> ON THE EXAM
>
> A health check path that touches the database turns a database outage into an empty target group. A path that touches nothing hides real failures. Choose deliberately.

The error codes, as a map

CODE	WHAT IT MEANS
400	Malformed request from the client. Not your target's fault
403	Blocked by AWS WAF, or by a rule returning a fixed response
460	The client closed the connection before the load balancer could respond
463	X-Forwarded-For header contained more than 30 IP addresses
500	An internal error inside the load balancer itself
502	The target returned a malformed response, or closed the connection
503	No healthy targets. The target group is empty or all targets failed
504	The target did not respond within the idle timeout

ON THE EXAM

460 is the one people misdiagnose. It means the client gave up, so look at client-side timeouts, not at your targets.

Where each code comes from

01

Client

460: the client gave up first. 400: it sent something malformed.

02

Load balancer

403 from WAF. 500 from the load balancer itself.

03

Target selection

503: there was no healthy target to choose.

04

Target connection

502: the target answered badly or hung up.

05

Target response

504: nothing came back before the idle timeout.

ON THE EXAM

Walk the path left to right and the code tells you exactly where to stop and look.

502, 503 and 504 in detail

- **502 Bad Gateway.** A target exists and answered wrongly: it crashed mid-response, returned malformed headers, or closed the connection.
- **503 Service Unavailable.** No healthy target. Check target health, the health check path, and the target security group.
- **504 Gateway Timeout.** The target was too slow. Default ALB idle timeout is **60 seconds**.
- For 504, raise the idle timeout **only if the work genuinely takes that long**. Otherwise fix the slow query, or make the work asynchronous.

> ON THE EXAM
>
> 502 means someone answered badly. 503 means nobody answered. 504 means nobody answered in time. That one sentence separates all three.

Draining, stickiness and slow start

- **Deregistration delay**, also called connection draining, defaults to **300 seconds**. In-flight requests finish before the target is removed.
- Too long and deployments crawl. Too short and users see errors mid-request. Match it to your longest normal request.
- **Stickiness**: a load balancer cookie for a duration, or an **application-based cookie** the target controls. NLB uses source IP.
- **Slow start** ramps traffic to a new target over 30 to 900 seconds, so a cold cache or JIT warm-up does not get hit at full rate.

> ON THE EXAM
>
> Slow start and Auto Scaling's instance warm-up solve the same problem at different layers. Slow start controls traffic; warm-up controls metrics.

Cross-zone load balancing, and its bill

- **ALB: always on** at the load balancer level, and **free**. It can be turned off per target group.
- **NLB: off by default**, and when enabled you are **charged for inter-AZ data transfer**.
- With it off, each zone's load balancer node only sends to targets in **its own zone**. Uneven target counts mean uneven load.
- Off is not a bug. It keeps traffic zonal, which lowers cost and limits the blast radius of a failed zone.

> ON THE EXAM
>
> "Two instances in one AZ are overloaded while eight in another are idle" behind an NLB is cross-zone load balancing being off. That is the default.

TLS and authentication

- Terminate TLS at the load balancer with a certificate from **ACM**. ACM certificates **renew themselves** when used this way.
- **SNI** lets one listener serve many certificates, so multiple domains share one load balancer.
- **Security policies** choose which TLS versions and ciphers are accepted. This is how you drop TLS 1.0 and 1.1.
- ALB can **authenticate users** via Cognito or any OIDC provider, and supports **mutual TLS** and **JWT validation** for machine-to-machine traffic.

> ON THE EXAM
>
> "The certificate expired and nobody renewed it" cannot happen with an ACM certificate attached to a load balancer. That is a strong signal for ACM.

What the load balancer can tell you

- **Access logs** to S3 record every request: client, target, latencies broken into three parts, and the status codes.
- Both the **ELB status code** and the **target status code** are logged, which separates a load balancer 502 from a target's own 500.
- **Connection logs** record TLS handshakes and client connections, including failures that never became a request.
- **Access logs are off by default**, and they are **not real time**. Files are delivered every 5 minutes.

> ON THE EXAM
>
> TargetResponseTime, plus target_status_code in the access log, is how you prove the slowness is the application rather than the network.

How targets are chosen

- **Round robin** is the ALB default, and it ignores how busy each target already is.
- **Least outstanding requests** sends to the target with fewest in-flight requests. Better when request cost varies a lot.
- **Weighted random** supports **automatic target weights**, which shifts traffic away from targets showing elevated errors.
- **Weighted target groups** split traffic between two groups by percentage, which is how you do blue/green and canary on an ALB.

> ON THE EXAM
>
> Weighted target groups are the ALB deployment mechanism. Module 22 uses them for blue/green; note the name now.

New: ALB Target Optimizer

- **Generally available 20 November 2025.** Enforces a **maximum number of concurrent requests per target**.
- Built for **compute-intensive, low-concurrency workloads** such as model inference, where a target may handle only one to five requests at a time.
- Enabled with a **target control port** on a new target group, plus an **AWS-provided agent** running on the targets.
- Traffic through such a target group **generates more LCU usage**, so it is not free.

> ON THE EXAM
>
> Without it, a hot-spotted target receives more than it can process, clients get errors and retry, and the retries add load. Target Optimizer removes that loop.

Capacity, and getting out of a bad zone

- **Load Balancer Capacity Unit Reservation** pre-provisions a minimum capacity ahead of a known surge. ALB and NLB.
- Consider it when you expect traffic to **more than double in under five minutes**: a ticket sale, a product launch, a broadcast.
- **Zonal shift** moves traffic away from one impaired Availability Zone, through Amazon Application Recovery Controller.
- **Zonal autoshift** lets AWS do it automatically when it detects a problem. Both are **off by default**.

> ON THE EXAM
>
> Load balancers scale automatically for organic growth. Reservation exists for the step change that outruns the scaling rate.

NLB and GWLB specifics

- **NLB gives a static IP per Availability Zone**, and you can attach your own Elastic IP. That is what firewall allow-lists need.
- **Source IP is preserved** to instance targets, so the application sees the real client address without X-Forwarded-For.
- **NLB TCP idle timeout is 350 seconds** and connections idle longer are dropped. Enable TCP keepalive for long-lived connections.
- **GWLB** uses **GENEVE on port 6081** to send traffic transparently through third-party firewalls and IDS appliances.

> ON THE EXAM
>
> "Our partner will only allow-list fixed IP addresses" is an NLB or Global Accelerator answer. An ALB's addresses change.

> After a deployment, an ALB begins returning HTTP 503 to all requests. The Auto Scaling group shows six instances in the InService state and CloudWatch shows normal CPU on all of them. Application logs show the service started successfully on each instance.
>
> The target group shows all six targets as unhealthy, with the reason 'Health checks failed'.

What is the most likely cause?

A The ALB's idle timeout is too low for the application

B The health check path or port changed, or the target security group does not allow the ALB security group on the health check port

C Cross-zone load balancing is disabled on the target group

D The deregistration delay is set too high

◆ PAUSE AND DECIDE

What is the most likely cause?

A **Idle timeout too low**

An idle timeout problem produces 504, not 503. The codes are not interchangeable, and that is the point of the table.

B **Health check path, port or security group**

503 means no healthy target. The application is running, so the check itself is failing: wrong path, wrong port, or the ALB cannot reach it.

C **Cross-zone disabled**

Causes uneven distribution, not universal failure. It is also always on for an ALB at the load balancer level.

D **Deregistration delay too high**

Affects how long removed targets keep draining. It cannot make registered, running targets fail their health checks.

THE SIGNAL

503 plus a running application means the health check disagrees with reality. Path, port, and the security group between ALB and target, in that order.

A reporting endpoint behind an ALB runs a query that legitimately takes about 90 seconds. Users receive HTTP 504 after roughly a minute. The targets are healthy and the query completes successfully when run directly on the instance.

The business wants the report to keep working, and the engineering team wants to avoid long-running HTTP requests.

Which two actions together are the best response? (Choose two.)

A. Increase the ALB idle timeout above 90 seconds as an immediate mitigation

B. Re-architect the report to run asynchronously, returning a job ID and polling for the result

C. Enable stickiness on the target group

D. Switch the target group's algorithm to least outstanding requests

◆ PAUSE AND DECIDE

Which two actions together are the best response?

A Raise the idle timeout

The default 60 seconds is exactly where the failure occurs. Raising it restores the report today, which the business asked for.

B Make the report asynchronous

Removes the long-lived HTTP request entirely, which is what the engineering team asked for. The proper fix, on a longer timescale.

C Enable stickiness

Binds a client to one target. It has nothing to do with how long a single request takes.

D Least outstanding requests

Changes which target is chosen. The chosen target is not the problem; the duration of the work is.

THE SIGNAL

Two stakeholders, two answers. "Choose two" questions often pair an immediate mitigation with the structural fix, and both are required.

> An NLB fronts a fleet spread across three Availability Zones: 2 instances in eu-west-1a, 8 in eu-west-1b, and 8 in eu-west-1c. The two instances in eu-west-1a run consistently at 90 percent CPU while the others sit near 25 percent.
>
> Health checks pass on every instance and the Auto Scaling group is configured across all three zones.

What explains this, and what is the trade-off in fixing it?

A. Cross-zone load balancing is off by default on an NLB. Enabling it evens the load, but adds inter-AZ data transfer charges

B. The target group is using round robin. Switch to least outstanding requests

C. Sticky sessions are pinning clients to eu-west-1a

D. The Auto Scaling group needs AZ rebalancing enabled

◆ PAUSE AND DECIDE

What explains this, and what is the trade-off in fixing it?

A Cross-zone off by default on NLB

Each zone's node serves only its own zone, so 2 instances absorb a third of the traffic. Enabling it is correct, and inter-AZ transfer is charged.

B Least outstanding requests

An ALB algorithm. It also would not help, because the imbalance is between zones rather than within one.

C Sticky sessions

Would concentrate individual clients, not produce a clean per-zone split matching the instance counts.

D ASG AZ rebalancing

The genuinely useful alternative: even out instance counts per zone. It fixes the symptom without the transfer cost, but the scenario asks what explains it.

THE SIGNAL

The numbers give it away. Three zones, traffic split three ways, and one zone holding a fifth of the instances. That is zonal routing, not an algorithm.

> A partner will only connect to the company's API if the endpoint has fixed IP addresses that they can add to their firewall. The API also needs path-based routing, because /v1 and /v2 are served by different target groups, and it must sit behind AWS WAF.
>
> The team has proposed giving the ALB an Elastic IP.

What should the team do?

A. Assign Elastic IPs to the ALB as proposed

B. Put an NLB in front of the ALB, using an ALB-type target group, and attach Elastic IPs to the NLB

C. Replace the ALB with an NLB and move path routing into the application

D. Publish the ALB's current IP addresses to the partner and update them when they change

◆ PAUSE AND DECIDE

What should the team do?

A Elastic IPs on the ALB

Not possible. An ALB does not support Elastic IPs; its addresses are managed by AWS and change as it scales.

B NLB in front of the ALB

The NLB supplies the static addresses, the ALB keeps path routing and WAF. The ALB-type target group is the supported way to join them.

C NLB only, routing in the app

Gets static IPs by throwing away path routing and WAF, since WAF does not attach to an NLB. It solves one requirement by breaking two.

D Publish and update addresses

The addresses change without warning as the ALB scales, so this is an outage waiting to be scheduled by AWS rather than by you.

THE SIGNAL

Three requirements that no single load balancer satisfies. The layered pattern exists precisely for this combination.

A machine learning inference service behind an ALB returns intermittent 502 errors under load. Each target can process about two requests concurrently before its queue builds and the process is killed by its own watchdog. Adding targets reduces but does not eliminate the errors, and clients retry, which increases load further.

The team wants to stop targets being sent more work than they can handle.

What should the team implement?

A ALB Target Optimizer, capping concurrent requests per target

B Increase the ALB idle timeout so slow requests are not cut off

C Enable slow start on the target group

D Switch the routing algorithm to round robin

◆ PAUSE AND DECIDE

What should the team implement?

A ALB Target Optimizer

Built for exactly this: low-concurrency compute-intensive targets. Caps concurrent requests per target, so a target is never over-committed.

B Raise the idle timeout

Addresses 504, not 502. The targets are dying, not being cut off, so a longer wait changes nothing.

C Slow start

Ramps traffic to a newly healthy target. Once the target is warm it receives full load again, and the problem returns.

D Round robin

The default, and it is unaware of concurrency. It is the algorithm that allows the over-commitment in the first place.

THE SIGNAL

The retry loop is the giveaway. Overloaded targets error, clients retry, retries add load. Only capping concurrency breaks that cycle.

The five things to carry forward

- **502 answered badly, 503 nobody answered, 504 nobody answered in time.** 460 means the client gave up.
- **503 with a running application** is the health check: path, port, or the security group between ALB and target.
- **Cross-zone is always on and free for ALB, off and charged for NLB.** Uneven zonal load is that default.
- **Static IPs plus path routing means NLB in front of ALB.** An ALB cannot take an Elastic IP.
- **Target Optimizer caps concurrency per target**, for compute-heavy workloads that handle only a few requests at once.

> ON THE EXAM
>
> Next: Multi-AZ, failover and health checks - RDS, Aurora, EFS and Route 53.